数字电路与逻辑设计实验（下）

题目 4：简易拼图游戏设计与实现

课程名称：数字电路与逻辑设计实验（下）

实验学期：2026～2027 学年第一学期

专业班级：电子工程 / 计算机科学与技术

学生姓名：课题组成员

学生学号：202300000000

指导教师：实验指导教师组

实验成绩：【评定中】

报告日期：2026 年 10 月

目 录

一、设计课题的任务要求（5 分） 1
1.1 课题背景与开发平台 1
1.2 任务规范与技术要求分解 1
1.3 硬件引脚分配规范与关键技术裁决 2
二、系统设计（20 分） 3
2.1 总体设计思路与微架构原则 3
2.2 系统总体框图与子系统划分 3
2.3 分块设计与模块接口一览 4
2.4 主状态机 ASM 设计 6
2.5 核心拼图引擎微架构 7
三、仿真波形及波形分析（25 分） 8
3.1 验证体系与 184 条断言全覆盖 8
3.2 模块实测波形深入分析 9
四、实际实现功能说明及资源利用情况（5 分） 13
4.1 实际功能指标达成 13
4.2 资源利用率极限量化分析 13
五、故障及问题分析（15 分） 15
5.1 典型工程缺陷六维深度剖析 15
5.2 典型工程缺陷排查对照汇总表 16
六、总结和结论（10 分） 17
6.1 课题完成情况评估 17
6.2 硬件 RTL 工程心得 17
6.3 未来展望 17
七、AI 工具使用情况（10 分） 17
7.1 工具栈与协同模式 17
7.2 真实 AI 提示词与交互实录 18
7.3 人工介入与纠错闭环 18
八、顶层文件的 HDL 代码（10 分） 19
8.1 顶层设计架构说明 19
8.2 规范注释的顶层 VHDL 代码清单 19

一、设计课题的任务要求（5 分）

1.1 课题背景与开发平台

本课题要求使用纯硬件描述语言（VHDL-93），在 Altera MAX II EPM1270T144C5 CPLD 上设计并实现一套高完整度、强交互性的简易拼图游戏系统。核心目标是在零软核/零微控制器的纯硬件电路上，完成自检、待机、目标预览、伪随机散落、平移旋转、碰撞检测、锁定与全图判定等全部游戏逻辑。

- 核心逻辑器件：Altera MAX II EPM1270T144C5（含 1,270 个逻辑单元 LE，127 个逻辑阵列块 LAB，无片上嵌入式 RAM 块）；
- 板载外设资源：50 MHz 有源晶振 (PIN_18)、使能开关 SW7 (PIN_125)、复位按键 BTN0 (PIN_61)、4×4 矩阵键盘、8×8 双色 LED 点阵屏、8 位共阴极数码管、电磁式无源蜂鸣器 (PIN_60)；
- EDA 工具链与环境：Quartus II 9.1 (32-bit，含内置仿真器与 TimeQuest 静态时序分析)、Python 3.14（12 套测试台 `sim/tb_*.py` 共 7,700 余行判据 + 23 个工具脚本 `scripts/*.py` 共 8,100 余行，合计约 1.6 万行自动化激励/判据与资源对账脚本）。本机未安装 ModelSim / GHDL，全部功能仿真均由 Quartus 内置仿真器完成（结果回写进 `sim/*.vwf`）。

1.2 任务规范与技术要求分解

课题涵盖 11 项基础功能要求（B1～B11）与 5 项自主提高要求（A1～A5）：

1. B1 使能与自检：SW7=‘0’时全系统低功耗静默熄灭、所有显示器件不显示；SW7=‘1’时开机自检——点阵 64 点全亮黄色并以 2 Hz、占空比 50% 闪烁，数码管全显“88888888”并同步闪烁，蜂鸣器奏响 POST 开机自检短鸣；自检窗口 2 s 后进入待机（B2）；
2. B2 待机状态：自检结束后点阵熄灭，数码管 DISP7 显示‘5’，DISP0 显示初始关卡‘1’，其余数码管熄灭；
3. B3 关卡指示：由最右侧数码管 DISP0 实时指示当前关卡（1/2/3）；本游戏基础为两关（第三关属 A2 扩展），且所有控制键统一使用 4×4 矩阵键盘；
4. B4 目标预览：待机按【开始】进入 5 秒预览，点阵红光显示目标图案（第一关为【题】图 4-1——课程题目 PDF 第 9 页——所示的 4×3 实心矩形，共 12 格），DISP7 倒计时 5～1；
5. B5 随机散落与对局：5 秒后自动散落，各零片随机分布且两两互不重叠（第一关 3 块零片，3+6+3=12 格）；DISP4:DISP3 倒计时，第一关限时 30 s；
6. B6 零片选择：按【选择】键循环切换，当前选中的零片高亮变绿，其余未锁定零片保持红色；
7. B7 平移与碰撞检测：按【上/下/左/右】平移，硬件实时拦截移出 8×8 边界或与其它零片重叠的非法指令；
8. B8 确认与锁定：按【确认】键锁定当前零片，锁定后零片变为黄色且不可再被选择及移动；
9. B9 成功与失败判定：全部锁定且画面与目标完全一致判定过关，数码管显示 PASS 并奏胜利乐句；超时或拼错判定失败，显示 FAIL 并奏失败乐句；结算画面先以 2 Hz 闪 2 个周期（1 s）后常亮，便于远距离判读；
10. B10 第二关：规则参考第一关，完整拼图图案自拟（本项目固定为 `PAT3` 阶梯），预览 5 s 后出现四块异形零片（1×3 横条 3 格 + 1×2 竖条 2 格 + J 形 5 格 + 六格块 6 格，共 16 格，位置随机、互不重叠），限时 40 s；成功显示胜利图案（绿色粗对勾），超时或拼错显示失败图案（红色叉）；
11. B11 重开局：对局中或结束后再次按【开始】即可开始新一轮（`game_fsm` 的 `S_PLAYING` / `S_WIN` / `S_FAIL` 三态都响应 `K_START`）；系统脱机上电即玩，无需任何上位机通信；
12. A1 丰富音效（附加）：定制马里奥风格背景乐（三关三曲）与自检 / 预览 / 过关 / 拼错 / 通关 / 失败事件乐句；对局期按键刻意静音（按上板实测反馈“按键音会打扰背景音乐”去掉按键与碰撞音，同时省 −9 LE），声音提示全部由背景乐与事件乐句承担；
13. A2 多关卡挑战（附加）：在 B10 第二关之后增加第三关（60 s，16 格，图案库随机抽取），支持三关连闯；
14. A3 位置随机与方向随机（附加）：零片位置随机已实现（`rng_lfsr` + 拒绝采样重试）；开局朝向随机按明确取舍未做 —— 改用固定非零初值（`PIECE_ORI_INIT = 1/1/2/2`，由 `scripts/check_geometry.py` 对第一关与第二/三关四幅图案穷举验证“不旋转则无解”），使【旋转】成为通关必需的一步，朝向由玩家自行旋转掌控；
15. A4 零片顺时针旋转（附加）：按【旋转】键在无碰撞情况下实现局部 90° 原位顺时针旋转；

16. A5 自拟其他功能（附加）：S1 第三关图案库随机 / S2 零片位置随机不重叠 / S3 分场景音效 / S4 第二关异形零片 / S5 结算画面先闪 2 周期再常亮（绿对勾 / 红叉）/ S6 零片 90° 旋转 / S7 倒计时 ≤10 s 的绿色 2 Hz 告急幽灵。

1.3 硬件引脚分配规范与关键技术裁决

信号名称（顶层端口）	FPGA 引脚	方向	电气标准	外设硬件映射及驱动逻辑说明
`clk`	PIN_18	输入	3.3V LVTTL	板载 50 MHz 有源晶振系统时钟主输入
`sw7`	PIN_125	输入	3.3V LVTTL	全局使能开关，‘0’=强制低功耗静默，‘1’=系统运行
`btn`	PIN_61	输入	3.3V LVTTL	全局复位按键 BTN0（板卡手册：空闲低、按下高，即按下为高有效）；`clk_gen` 用 20 ms 饱和计数器同步消抖，再与上电复位相或得高有效 `o_rst`
`kp_row[3..0]`	PIN_114, 113, 112, 111	输入	3.3V LVTTL	4×4 矩阵键盘行输入（主控内部上拉，低有效）
`kp_col[3..0]`	PIN_120, 119, 118, 117	输出	3.3V LVTTL	4×4 矩阵键盘列扫描（独热低电平循环轮询）
`dot_row[7..0]`	PIN_1, 2, 3, 4, 5, 6, 7, 8	输出	3.3V LVTTL	8×8 点阵行驱动（低电平选通，每次只选一行）
`dot_colr[7..0]`	PIN_11, 12, 13, 14, 15, 16, 21, 22	输出	3.3V LVTTL	8×8 点阵红色列驱动（高电平点亮红色 LED）
`dot_colg[7..0]`	PIN_38, 39, 40, 41, 42, 43, 44, 45	输出	3.3V LVTTL	8×8 点阵绿色列驱动（高电平点亮绿色 LED）
`cat[7..0]`	PIN_31, 30, 70, 69, 68, 67, 66, 63	输出	3.3V LVTTL	8 位数码管位选（共阴极低有效驱动）
`seg[7..0]`	PIN_51, 52, 53, 55, 57, 58, 59, 62	输出	3.3V LVTTL	数码管段选 a～g、dp（共阴极高有效驱动）
`buzz`	PIN_60	输出	3.3V LVTTL	无源电磁式蜂鸣器方波驱动输出

以上 11 类信号共锁定 52 个用户 I/O，与 `Fitter Summary` 的 `Total pins 52 / 116 ( 45 % )` 逐脚吻合；`.pin` 报告中不存在任何“未锁定/自动分配”的用户引脚（`RESERVE_ALL_UNUSED_PINS = AS INPUT TRI-STATED`，其余 64 个未用脚保持高阻）。

硬件设计关键裁决：

- 裁决 A（矩阵键盘物理布局）：统一采用 4×4 矩阵键盘，按板卡手册附图 26 实测定稿 8 个功能键：`KEY14` =开始、`KEY16` =选择、`KEY11` =确认、`KEY7` =上、`KEY15` =下、`KEY10` =左、`KEY12` =右、`KEY8` =旋转；复位由独立按键 BTN0 承担，不占矩阵键位，因此无需静音键（对局期本就不发按键音）。译码集中在 `game_fsm.key_of()` 一处（箭头左侧为键盘扫描码、右侧为 `puzzle_pkg` 里的键常量名，两者数值不同：`"0001"→K_START`、`"0011"→K_SELECT`、`"0110"→K_CONFIRM`、`"1010"→K_UP`、`"0010"→K_DOWN`、`"0101"→K_LEFT`、`"0111"→K_RIGHT`、`"1011"→K_ROT`），其余键返回 `K_NONE`，彻底解决板载独立微动按键不足的问题；
- 裁决 B（双色点阵显色逻辑）：两列均为高电平点亮——仅红（`dot_colr='1'`、`dot_colg='0'`）为未选中/游离零片；仅绿（`dot_colg='1'`）为当前选中零片及告急幽灵（≤10 s 目标图案绿色闪烁提示）；两列同时为‘1’时该点点亮两个 LED，呈黄色，即已正确锁定的零片；两列全 ‘0’ 则该点熄灭；

- 裁决 C（数码管空间映射与冲突消解）：实验说明书第 3 条要求关卡由最右侧 DISP0 指示，而第 9 条又提“进入第二关，数码管 DISP2 显示 2”。经工程论证，若点亮 DISP2 会破坏倒计时视觉排布（DISP4:DISP3）且第三关无对应规则。工程坚决执行第 3 条全局规则：全关卡统一由 DISP0 指示关卡，DISP2 在第二关及对局中恒定熄灭。

二、系统设计（20 分）

2.1 总体设计思路与微架构原则

针对 Altera MAX II EPM1270T144C5 仅有 1,270 个逻辑单元 (LE) 且无嵌入式 RAM 块的极限资源约束，设计确立了三项微架构设计原则：

1. 同步单时钟域与使能脉冲节拍驱动：全系统仅在 PIN_18 50 MHz 主时钟上升沿驱动，严禁在分频时钟上挂载触发器。通过一条级联分频链产生 1 kHz / 200 Hz / 100 Hz / 40 Hz / 4 Hz / 2 Hz / 1 Hz 七路单周期、同源无毛刺使能脉冲（每条使能只占 1 个 20 ns 主时钟周期）；
2. 64 位并行位平面与纯组合空间运算：利用 CPLD 宽布线优势，将 8×8 拼图画面表达为 64 位 `std_logic_vector`，平移与旋转为单拍 64 位位运算；重叠检测按“候选行 × 邻块”串行扫描（单次 5～16 拍，即 100～320 ns）；全图判据按行切片累积、在几何帧末（第一关 40 ms、第二/三关 48 ms）发布——以少量串行换取面积，杜绝多周期状态机与时钟膨胀。（术语约定：本报告中未加限定的「帧」一律指几何帧，即拼图引擎把 64 位画面按 8 行切片比对一遍的 40/48 ms 周期；显示侧的「扫描帧」会明确写出，指点阵 8 行轮扫一遍的 8 ms 周期。）
3. 纯组合查找表替代复杂算术核：倒计时的 /10 与 %10 展开为 64 项常量表（`disp_format` 实测 17 LE）；散落取模改为“一次比较 + 条件相减”；全设计不含除法器与三角函数，从根上消除长组合路径的时钟约束风险。

2.2 系统总体框图与子系统划分

先看功能总览：它把 11 个 entity 按功能归成 6 个功能块 + 1 个时钟块，用来回答“数据怎么流、谁管谁”：

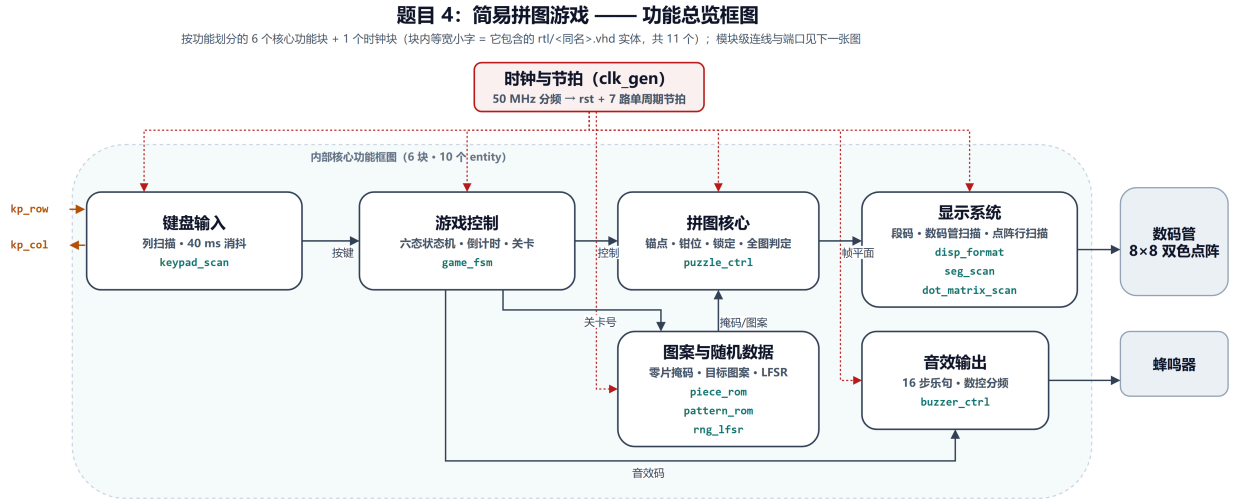


图 1　功能总览框图（子系统级）：6 个功能块 + 时钟块，每块框内的等宽小字是它包含的 `rtl/<同名>.vhd` 实体（11 个 entity 恰好划分一次，由脚本与 `facts.json` 对账）；红色虚线 = `clk_gen` 分发的全局时钟与节拍

整机划分为 5 大高层级核心子系统，其职责分工如下：

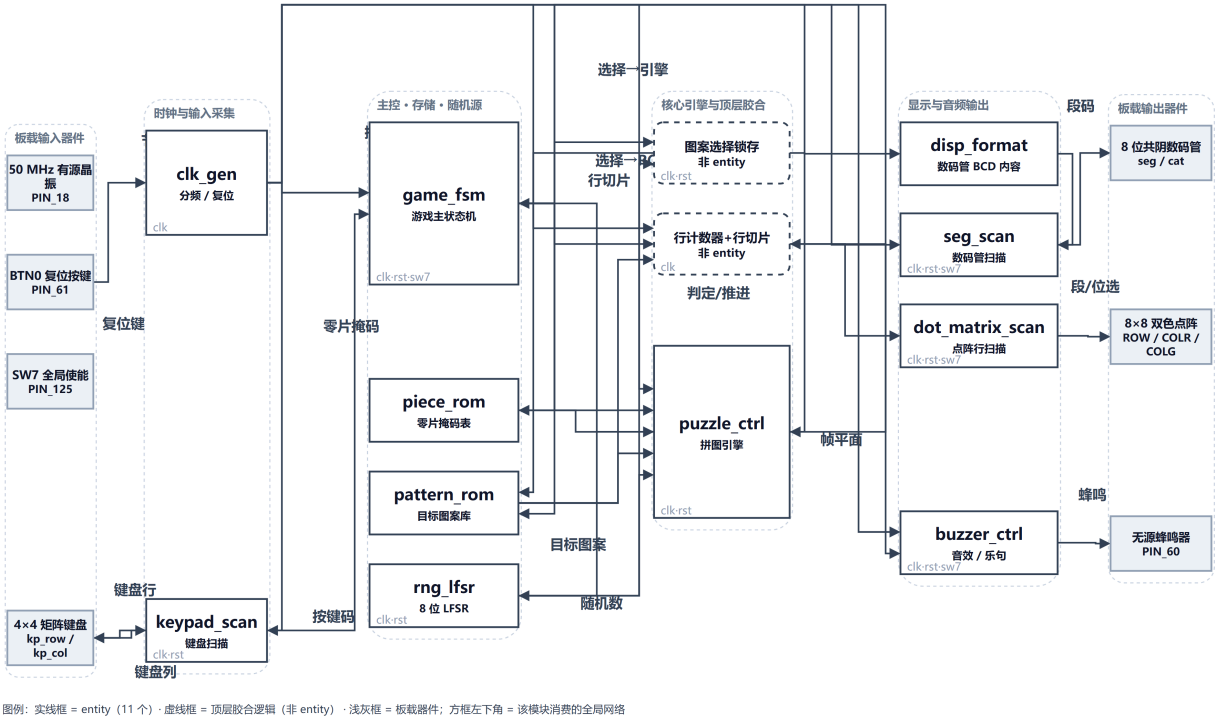
1. 时钟使能与键盘输入子系统：主频 50 MHz 经分频产生全局单周期 Tick 使能脉冲网络；4×4 矩阵键盘由 200 Hz 节拍驱动、两拍一轮（一轮 10 ms）的独热列释放扫描；`DEBOUNCE_MAX+1 = 4` 轮判稳 ⇒ 40 ms 消抖（8 个 200 Hz 采样点），随后向主控输出稳定的单拍按键脉冲；
2. 游戏主控与规则状态机：统一管理系统全局状态迁移（自检、待机、预览、对局、结算）、倒计时递减及连闯三关流转；
3. 拼图核心引擎：包含 8 位 LFSR（周期 255）伪随机散落算法、64 位几何平移与 90° 旋转、刚性碰撞越界拦截及全图 100% 吻合判定；

4. 图案与随机数据 ROM：固化关卡目标图案库与各零片几何掩码，并为拼图引擎提供无冲突的伪随机散落偏移量；
5. 显示与音效输出子系统：实现数码管与点阵屏的 125 Hz 超临界无频闪扫描驱动，叠加 ≤10 s 告急幽灵闪烁；音频模块内置数控分频与 16 步连奏音符发生器，输出马里奥经典音效。

再看模块级细节：11 个 entity 逐个成框、逐根端口连接如下图所示（各模块的职责与端口详见 §2.3）。读图时请注意三点：① 数据通路自左向右 —— 板载输入 → 时钟与键盘采集 → 主控与存储 → 核心引擎 → 显示/音频输出 → 板载输出；② 同一个源模块出去的线共用一条主干（一根线 = 一条总线），到各自目标再分叉，因此图上没有“32 根线各带一个标签”那种糊成一片的画法；每根主干到底走了哪几根线、多少位，逐条列在 §2.3 的模块接口表里；③ 全局网络 `clk`（← 50 MHz 有源晶振）、`rst`（← `clk_gen`）、`sw7` 使能（← `SW7`）是一对多的分配关系，逐根画会得到 20 根长线，故改用模块角标表达，消费者与来源在图中均可查。

题目 4：简易拼图游戏 —— 系统总体框图

每个实线框 = rtl/<同名>.vhd 中的一个 entity；同一个源模块出去的线共用一条主干，箭头旁为信号组名（网名与位宽见报告 §2.3）



图例：实线框 = entity（11 个）· 虚线框 = 顶层胶合逻辑（非 entity）· 浅灰框 = 板载器件；方框左下角 = 该模块消费的全局网络

图 2　系统总体框图（模块级）：实线框 = `rtl/<同名>.vhd` 中的一个实体（共 11 个，与 §8.2 的 11 条 `component` 声明、11 个 `u_*` 例化三方一致）；虚线框 = `puzzle_top.vhd` 内的顶层胶合逻辑；浅灰框 = 板载器件。同一个源模块出去的线共用一条主干（图中一根线即一条总线，到各自目标再分叉），箭头旁标的是信号组名，图上一共 32 条框间连接（每条聚合若干根网名，合计 60 个「源→目标→网名」三元组、对应 puzzle_top.vhd 的 127 行 port map），网名与位宽见 §2.3 的模块接口表；每个模块左下角的小字标出它消费的全局网络（`clk` / `rst` / `sw7`）

2.3 分块设计与模块接口一览

整机由 11 个子模块组成，全部采用具名端口映射例化（与 §8.2 的 `component` 声明逐一对齐）。下表给出各模块的职责、关键端口、驱动节拍与关键内部信号，作为 §8.2 顶层代码的阅读索引；「关键端口」一列写的是各模块自己的 `i_` / `o_` 端口名（与 `rtl/<模块>.vhd` 的 `entity` 声明逐字一致），`→` 左侧为输入、右侧为输出，`[m:n]` 为位宽：

模块	职责	关键端口（方向）	驱动节拍	关键内部信号 / 实现要点
`clk_gen`	级联分频链，产生 7 路单周期 tick	`i_clk`, `i_btn` → `o_rst`, `o_tick_1k` … `o_tick_1hz`（7 路）	50 MHz	`c1..c7` 级联链；`bcnt` 用 20 ms 饱和计数器对 BTN0 同步消抖
`keypad_scan`	4×4 矩阵扫描 + 消抖	`i_tick`, `i_row[3:0]` → `o_col[3:0]`, `o_key[3:0]`, `o_press`, `o_release`	`i_tick` = 200 Hz（两拍一轮 = 10 ms）	`cnt` 数 4 轮判稳（40 ms），松开立即撤销
`game_fsm`	主状态机 / 倒计时 / 音效事件码	`i_press`, `i_key[3:0]`, `i_sw`, `i_solved` → `o_state[2:0]`, `o_time[5:0]`, `o_blink`, `o_sound[3:0]`, `o_up..o_rot`	`i_tick_1hz` / `i_tick_4hz`	`st` 六态；`key_of()` 单点译码 8 个功能键
`puzzle_ctrl`	散落 / 平移 / 旋转 / 碰撞 / 全图判据	`i_go`, `i_select`, `i_move`, `i_confirm`, `i_rot`, `i_up..i_right`, `i_target[63:0]`, `i_sh0..i_sh3[63:0]`, `rnd_val[7:0]` → `o_rowr[63:0]`, `o_rowg[63:0]`, `o_solved`, `rnd_step`	`i_tick` = 1 kHz	`chk_slot` / `chk_crow` / `chk_hit`、`frm_ok`、`alllock_r`；几何帧 40/48 ms
`rng_lfsr`	8 位 Galois LFSR（$x^8+x^6+x^5+x^4+1$）	`i_step` → `o_val[7:0]`	使能	`sr`，种子 `x"5A"`，满周期 255
`piece_rom`	各关零片几何掩码 / 包围盒	`i_level` → `o_mask[×4]`, `o_h[×4]`, `o_w[×4]`, `o_n[2:0]`	组合	常量查找表，4 块并行输出；面积严格 3+6+3 / 3+2+5+6
`pattern_rom`	三关目标图案库	`i_level`, `i_pat[1:0]` → `o_mask[63:0]`	组合	第一关 1 幅 12 格 + 第二/三关库 4 幅 16 格，共 5 幅且全连通
`disp_format`	数码管 BCD 内容 / PASS · FAIL 字符拼装	`i_state[2:0]`, `i_time[5:0]`, `i_blink` → `o_data[31:0]`, `o_blank[7:0]`	组合	64 项常量表（消除除法器）
`seg_scan`	8 位数码管动态扫描	`i_data[31:0]`, `i_blank[7:0]` → `o_seg[7:0]`, `o_cat[7:0]`	`i_tick` = 1 kHz	`idx` 独热位选，段选与位选同拍寄存
`dot_matrix_scan`	8×8 双色点阵行扫描	`i_row[2:0]`, `i_colr[7:0]`, `i_colg[7:0]` → `o_row[7:0]`, `o_colr[7:0]`, `o_colg[7:0]`	行号由顶层 `mrow` 以 1 kHz 推进（本模块无 `i_tick`）	`row_sel` 低电平独热；行号与列数据同拍更新

模块	职责	关键端口（方向）	驱动节拍	关键内部信号 / 实现要点
`buzzer_ctrl`	数控分频 + 16 步乐句发生器	`i_en`, `i_sel[3:0]`, `i_t4` → `o_buzz`	50 MHz（÷256 预分频）	`pre_cnt` 半周期计数、`step` 乐句步进（换码不复位）

2.4 主状态机 ASM 设计

整机的决策流程如下图所示：圆角框为起止、矩形为处理、菱形为判断，判断分支标注 是 / 否；对局阶段是一个循环（图中用容器框圈出，三条回边汇入一条「回循环」总线）。

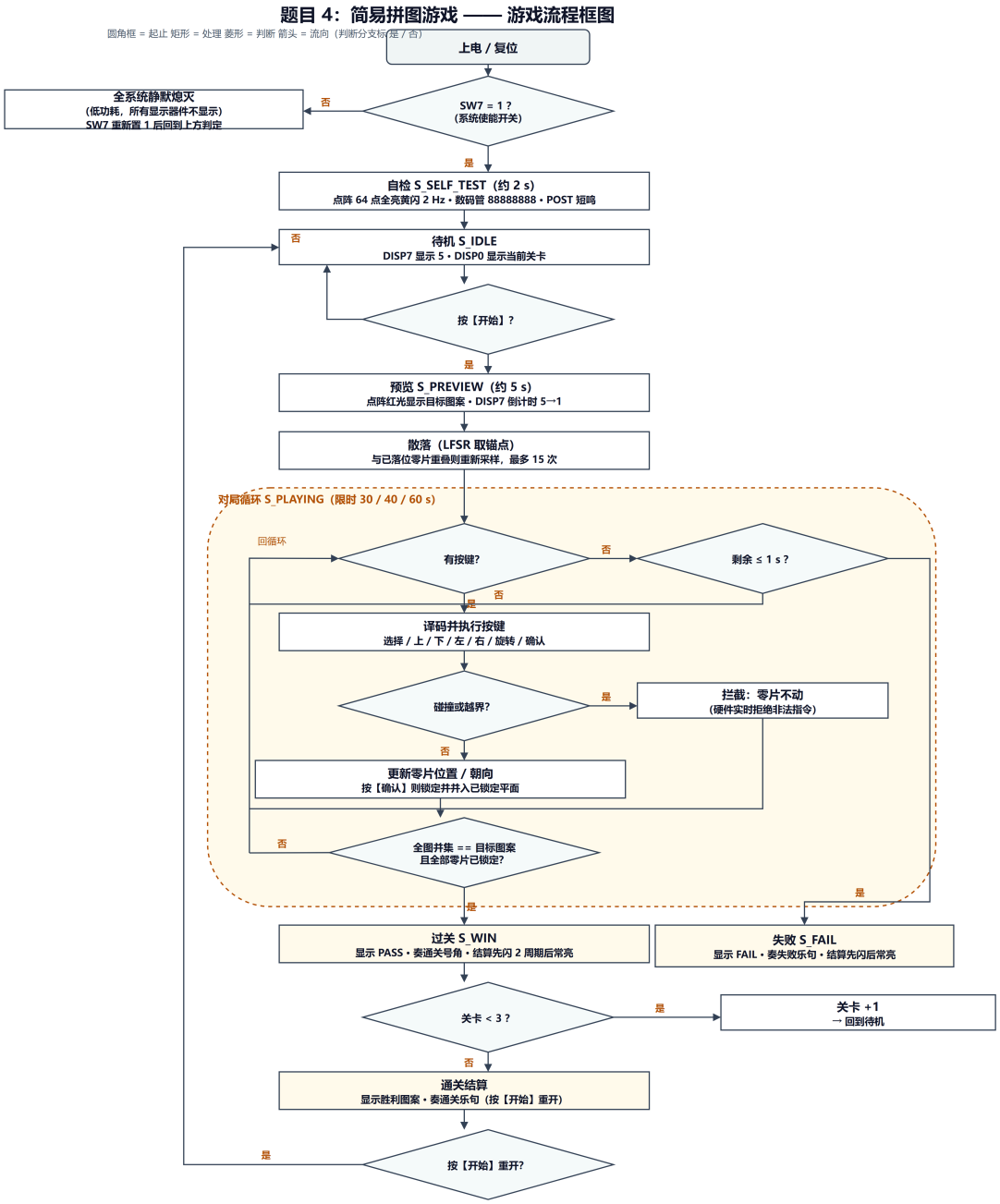


图 3　游戏决策流程框图（自检 → 待机 → 预览 → 对局循环 → 过关 / 失败 / 通关）

游戏主控状态机（FSM）严格遵循硬件算法状态机（ASM）设计规范，其状态转移与跳转条件如下图所示：

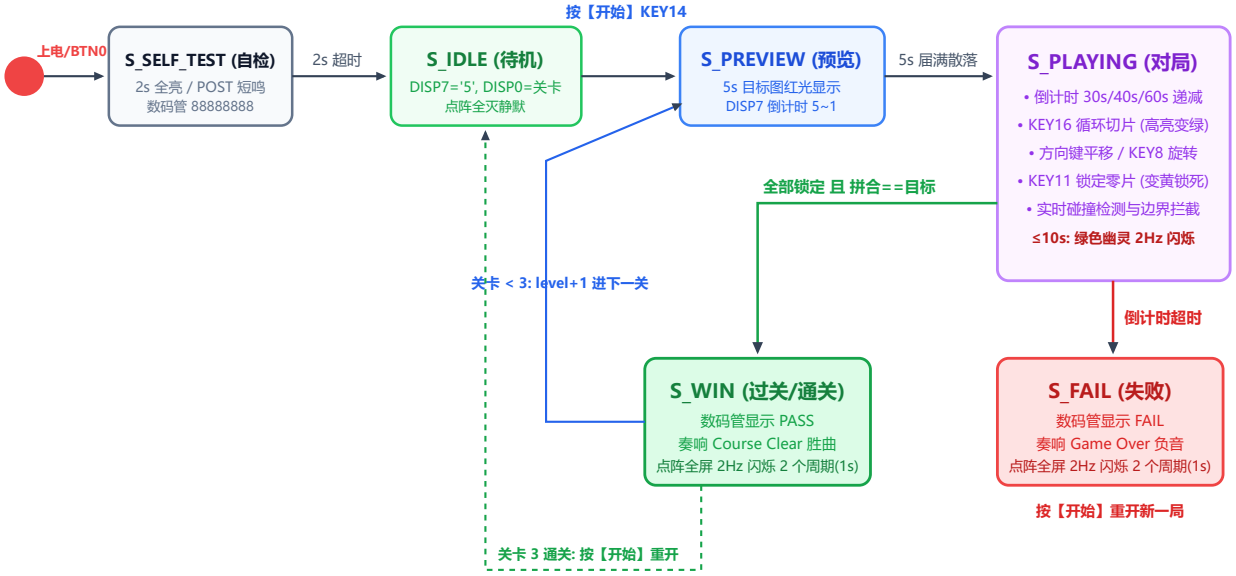


图 4　游戏主控状态机 ASM 状态转移流程图

状态机包含 6 个核心状态：

- S_SELF_TEST（自检）：上电或复位进入，窗口恒为 8 个 4 Hz 节拍（复位路径恰 2.000 s；SW7 关→开重入落在 1.75～2.0 s 之间），数码管显示 88888888，点阵 64 点全亮（红+绿=黄）并以 2 Hz、占空比 50% 闪烁，蜂鸣器奏响 POST 开机自检短鸣；
- S_IDLE（待机）：自检后点阵全熄灭，DISP7 显示‘5’，DISP0 显示当前关卡；按【开始】键触发预览；
- S_PREVIEW（预览）：点阵全红显示目标图案，DISP7 倒计时 5～1 s，限时届满自动散落；
- S_PLAYING（对局）：LFSR 散落各零片，倒计时（30 s / 40 s / 60 s）以 1 Hz 递减，响应平移与旋转。倒计时 ≤10 s 时触发 2 Hz 绿色告急幽灵闪烁辅助；
- S_WIN（过关）：全部零片锁定且拼合像素与目标 100% 重合时进入，数码管显示 PASS，奏响通关号角；结算画面先以 2 Hz 闪 2 个周期（1 s）后常亮。关卡 < 3 自动升关，第三关通关保持大通关状态，按【开始】重开新一局；
- S_FAIL（失败）：倒计时归零超时或错拼锁定进入，数码管显示 FAIL，奏响 Game Over 失败音；结算画面同样先闪 2 个 2 Hz 周期（1 s）后常亮并停留在结算画面，等玩家按【开始】重开新一局（不会自动返回待机）；

2.5 核心拼图引擎微架构

拼图核心引擎采用数据流解耦与并行计算微架构，其内部运算流程如下图所示：



图 5　拼图核心引擎数据流与微架构框图

引擎包含四大处理阶段：

1. 随机散落与锚点生成：`rtl/rng_lfsr.vhd` 采用 8 位 Galois 型 LFSR，反馈抽头为 `sr(7) xor sr(5) xor sr(4) xor sr(3)`，即本原多项式 $x^8+x^6+x^5+x^4+1$，复位种子 `x"5A"`，满周期 $2^8-1=255$；散落按零片包围盒取候选锚点：每次尝试推一步 LFSR，取其低 6 位（2×3 位两段）分别对行/列上界 `9 - hh` / `9 - ww` 作条件相减取模；若与已落位零片重叠则重新采样（最多 15 次），仍失败则落 4 个确定性兜底锚点之一：这 4 个锚点彼此不重叠，但不与已落位零片做重叠检查（已知缺陷 ERR-018b，仅随机源连续冲突时可能触发，如实未修）；
2. 64 位几何变换处理器：根据当前坐标 (x,y) 对零片 ROM 掩码执行二维位移；按下旋转键时，在局部包围盒内执行 90° 顺时针正交旋转；
3. 刚性碰撞拦截检测：零片本行占用 `chk_crow` 与其余各零片的本行掩码逐块相与（`chk_slot` = 邻块序号 `0..npc-1`，一拍扫一块），命中即置高 `chk_hit`；`chk_hit = '1'` 时当拍锁死坐标、禁止更新，越界同样拦截，确保物理刚性；
4. 锁定与全图验证：按【确认】后当前零片并入已锁定平面并置 `alllock_r`；全图判据为“所有零片并集 == 目标图案”，成立即置 `frm_ok`，再与 `alllock_r` 相与得到 `o_solved` —— 判据按行切片累积、在几何帧末发布（第一关 40 ms、第二/三关 48 ms，远快于人的操作节奏），不做“第 k 块必须在 (x_k, y_k)”式的坐标绑定。

三、仿真波形及波形分析（25 分）

3.1 验证体系与 184 条断言全覆盖

工程建立了“Quartus II 内置仿真器 + `.vwf` 激励/结果回写 + Python 自检判据”的自动化验证体系（本机没有 ModelSim / GHDL）：12 套 `sim/tb_<模块>.py` 覆盖 11 个子模块与 1 个整机顶层，累计 184 条判据断言全部通过。下表断言数与最新轮次逐行取自 `sim/rounds/<模块>/rNN.json`，与 `docs/03-仿真验证方案.md` §2 的对账表一致：

测试平台（`sim/tb_*.py`）	被测模块	断言条数	测试激励类型	关键验证项与边界覆盖范围
`tb_clk_gen`	`clk_gen`	10	周期计数穷举	50 MHz 经级联分频链得到 7 路（1 kHz / 200 Hz / 100 Hz / 40 Hz / 4 Hz / 2 Hz / 1 Hz）单周期使能脉冲的宽度与周期精确性
`tb_keypad_scan`	`keypad_scan`	9	按键反弹与时序抖动	40 ms 硬件消抖、矩阵行列反相译码、8 个功能键码的唯一译码、长按单脉冲输出
`tb_game_fsm`	`game_fsm`	30	主状态全流程穷举	SELF_TEST → IDLE → PREVIEW → PLAYING → WIN / FAIL 的转换边界、2 s / 5 s / 30 s / 40 s / 60 s 定时边界、连闯三关流转
`tb_puzzle_ctrl`	`puzzle_ctrl`	28	几何运算与空间移动	零片循环切换、上下左右边界拦截、多块重叠碰撞检测、局部 90° 顺时针旋转、主路径竞态
`tb_rng_lfsr`	`rng_lfsr`	10	255 拍序列穷举	本原多项式逐拍移位一致性、全零吸收态自锁保护、汉明重分布、复位重演
`tb_piece_rom`	`piece_rom`	10	几何掩码查表	第一关（3+6+3=12 格）与第二/三关（3+2+5+6=16 格）零片面积守恒、左上角归一化及旋转空间映射

测试平台（`sim/tb_*.py`）	被测模块	断言条数	测试激励类型	关键验证项与边界覆盖范围
`tb_pattern_rom`	`pattern_rom`	8	图案位图一致性	【题】图 4-1 矩形及田/十字/SZ/阶梯图案连通性、格数严格 12 / 16 格
`tb_disp_format`	`disp_format`	16	全字符译码映射	0～9 数字译码、前导零消隐、PASS/FAIL 拼装字符、倒计时 case 表 t=0..63 逐项
`tb_seg_scan`	`seg_scan`	12	动态扫描时序	8 位共阴极位选 125 Hz 轮巡、段选与位选同拍寄存（无独立消隐相）、`i_blank` 逐位熄灭、`i_en='0'` 全局静默
`tb_dot_matrix_scan`	`dot_matrix_scan`	7	双色点阵驱动	8 行独热低电平行选通、红/绿列直通（高电平点亮）、单色内容下红绿列互斥、`i_en='0'` 全灭、+20 ns 早采样点无混合态
`tb_buzzer_ctrl`	`buzzer_ctrl`	16	音频数控分频	E5～G6 音符分频精度（实测误差 0.59%～1.25%，容差 1.5%）、50% 占空比、连奏填音与“只有提示音允许休止”的设计规则自检
`tb_puzzle_top`	`puzzle_top`（整机）	28	真实用户按键全流程	模式 A / 模式 B 两轮真实按键序列，完成自检、待机、预览、通关三关、按【开始】重开后失败判定的端到端闭环，并校验告急幽灵“在闪且只闪绿色”、倒计时 11 s 时不得画幽灵

各模块最新轮次（同一份对账口径）：clk_gen `r25`、keypad_scan `r27`、game_fsm `r36`、puzzle_ctrl `r37`、pattern_rom `r21`、piece_rom `r23`、rng_lfsr `r23`、disp_format `r27`、seg_scan `r31`、dot_matrix_scan `r23`、buzzer_ctrl `r37`、puzzle_top `r56`（模式 A）/ `r57`（模式 B）。12 套测试台合计 184 / 184 全部通过。

3.2 模块实测波形深入分析

下列波形均为 Quartus II 9.1 波形编辑器界面的现场截图（`sim/<模块>.vwf` 由内置仿真器回写真实结果后打开），每张图单独占通栏宽度以保证信号名与时间刻度可读；每张图对应的逐条断言、实测值与激励时长见 `sim/rounds/<模块>/rNN.json` 与 `docs/03-仿真验证方案.md` §2 的对账表。

关于仿真时间尺度的如实说明：本设计的最低频节拍是 1 Hz（游戏秒计数），若按 50 MHz 主频原速仿真，仅一次 60 s 限时的第三关就需要约 30 亿个时钟周期，无法在可观测的时间窗内出图。因此整机与分频链的仿真对 RTL 施加了受控的、逐条留痕的时间压缩补丁：`clk_gen` 的分频常数按比例缩小（50,000,000 → 10,000），整机仿真的零片随机种子、关卡限时常量与图案选择被固定，使「自检 → 三关连闯 → 结算」的完整轨迹在约 125 ms 内跑完（对应 `sim/rounds/puzzle_top/r57.json` 记录的 `rtl_patches`）。因此波形横轴的绝对时间是被压缩过的，图上读到的是节拍之间的相对先后与相位关系；所有断言判据、端口时序关系与占空比结论都不依赖绝对时间。补丁清单随每轮记录入库（`rtl_patches` 字段），可逐条复核；清洁重编译与上板固件不含任何补丁。

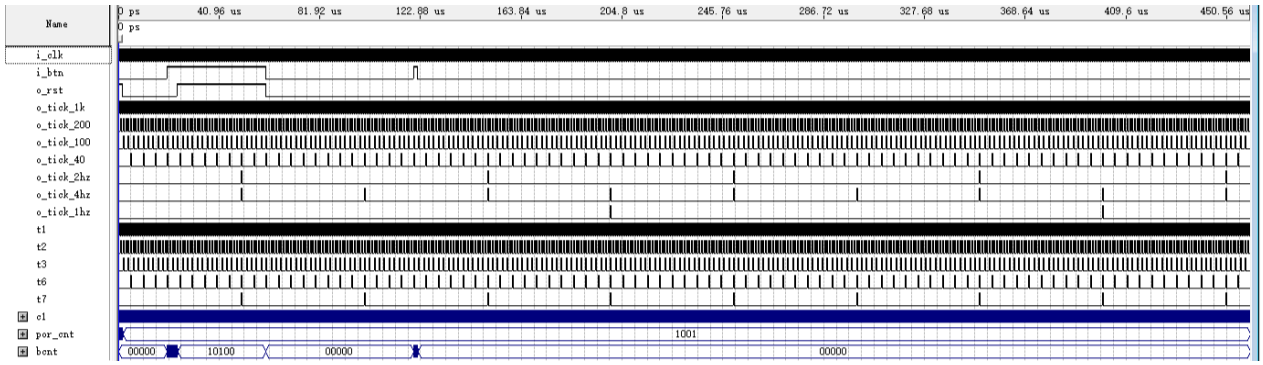


图 6　clk_gen 仿真波形：七路单周期使能脉冲（1 kHz / 200 Hz / 100 Hz / 40 Hz / 4 Hz / 2 Hz / 1 Hz）

时钟分频分析：主频 50 MHz 经级联分频链产生 7 路单周期使能——1 kHz（点阵行扫描 / 数码管位选 / 引擎行渲染）、200 Hz（键盘扫描轮次与消抖采样）、100 Hz / 40 Hz（板级自检诊断；100 Hz 当前无读者）、4 Hz（翻转得 2 Hz 方波：自检闪烁与结算闪示）、2 Hz（仅自检计数）、1 Hz（游戏秒计数）。每条使能严格维持 1 个主时钟周期（20 ns），同源无毛刺，杜绝门控时钟冒险。

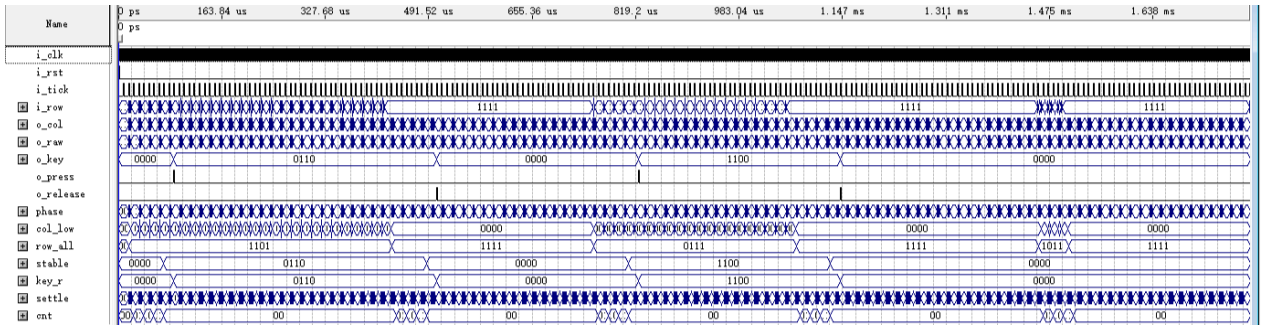


图 7　keypad_scan 仿真波形：列独热轮巡、40 ms 滞回消抖与单拍按键脉冲

键盘消抖分析：列输出 `kp_col[3:0]` 由 200 Hz 节拍驱动、两拍一轮（一轮 10 ms）的独热释放扫描。行输入在 5～15 ms 机械抖动期间不会立即响应：消抖计数器要求连续 4 轮（= 40 ms，8 个 200 Hz 采样点）读数一致才判稳，随后输出严格单时钟周期的按键脉冲，彻底消除连击。

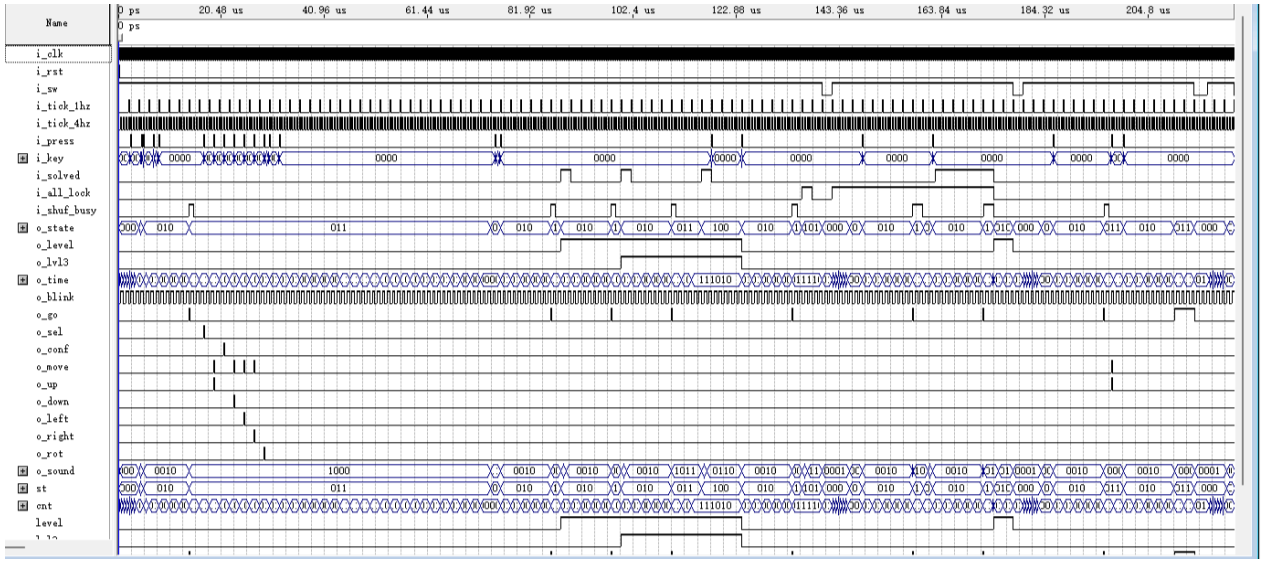


图 8　game_fsm 仿真波形：S_SELF_TEST → S_IDLE → S_PREVIEW → S_PLAYING → S_WIN 全流程

状态机迁移分析：展示完整轨迹：S_SELF_TEST（自检窗口 8 个 4 Hz 节拍，复位路径恰 2.0 s）→ S_IDLE（待机）→ S_PREVIEW（5 s 预览倒计时）→ S_PLAYING（散落零片并开启 30 s 倒计时）→ S_WIN（通关结算后跳向关卡 2）。

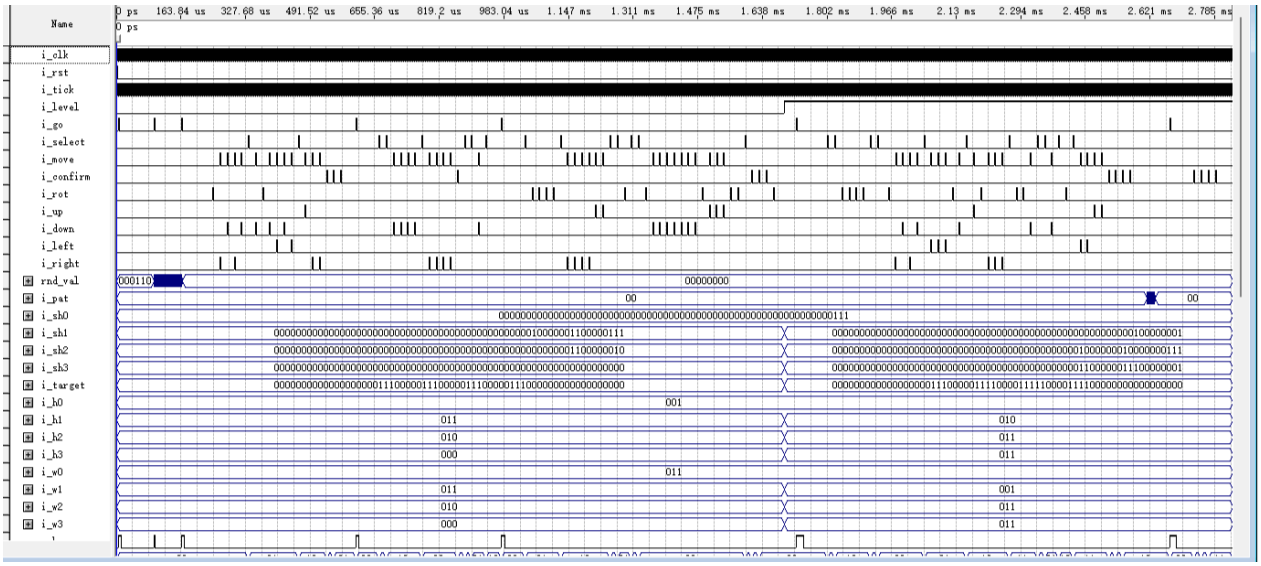


图 9　puzzle_ctrl 仿真波形：零片平移、边界拦截与锁定并入

拼图引擎运算分析：按下【左】键时零片正常平移；移动至边界（x = 0）再次按【左】时，碰撞检测逻辑瞬间置高内部信号 `chk_hit`，坐标当拍保持不变；按【确认】后零片并入已锁定平面（置 `alllock_r`）。

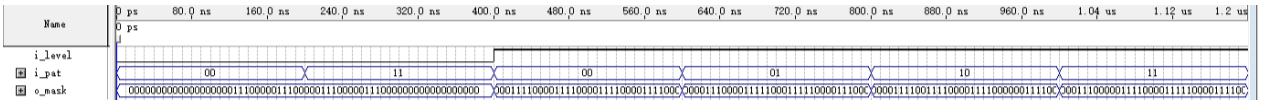


图 10　pattern_rom 仿真波形：三关目标图案库输出

目标图库分析：第一关恒定输出【题】图 4-1 实心矩形（12 格）；第二/三关依 `i_pat` 精准输出田 / 十字 / SZ / 阶梯 4 幅高难度图案，面积严格为 16 格且全连通无洞。

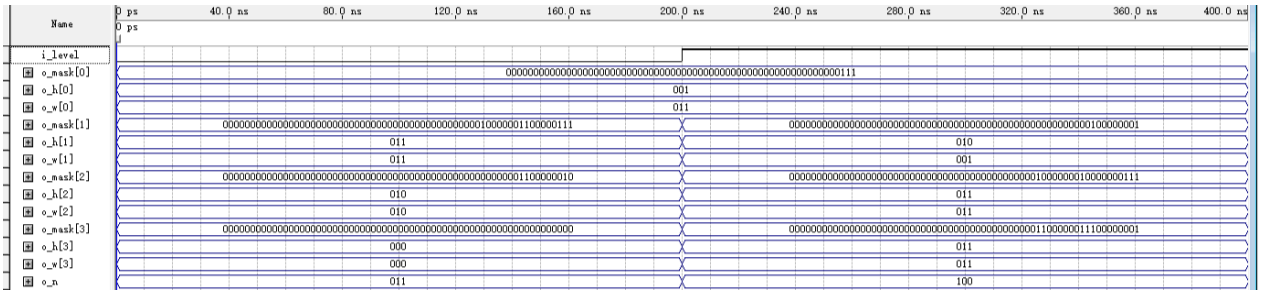


图 11　piece_rom 仿真波形：各关零片几何掩码

零片几何分析：第一关 3 块零片格数 3+6+3=12；第二关 4 块异形格数 3+2+5+6=16，面积守恒。全部零片完成左上角（0,0）归一化，具备精确可铺性。

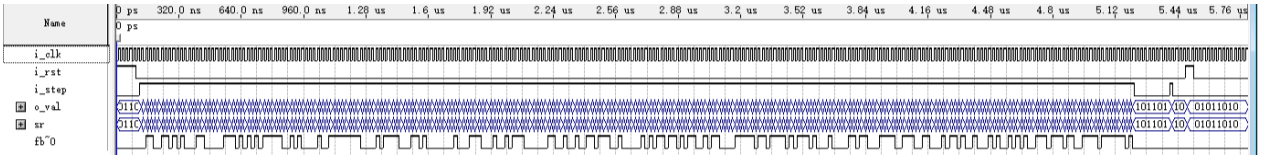


图 12　rng_lfsr 仿真波形：8 位 LFSR 的 255 拍满周期序列

随机数源分析：复位稳定于初值 0x5A。在 255 步时钟下状态与本原多项式逐拍一致，无全零吸收态，每位出现 ‘1’ 概率严格为 128/255，中途复位可靠回归初值。

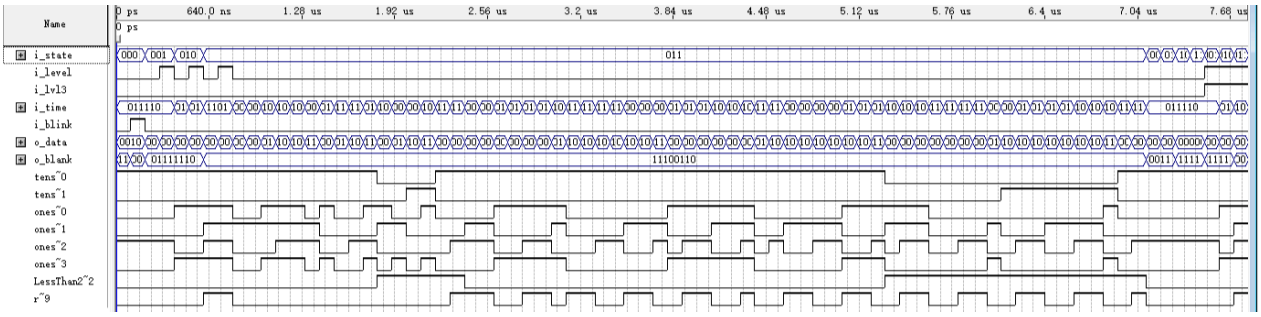


图 13　disp_format 仿真波形：数字译码、前导零消隐与 PASS/FAIL 拼装

显示格式化分析：64 项常量表实现倒计时十位/个位拆分，彻底消除 26 ns 除法器延时。除自检全 8 外，其余 76 个窗口 DISP2 恒定熄灭，胜利输出 PASS，失败输出 FAIL。

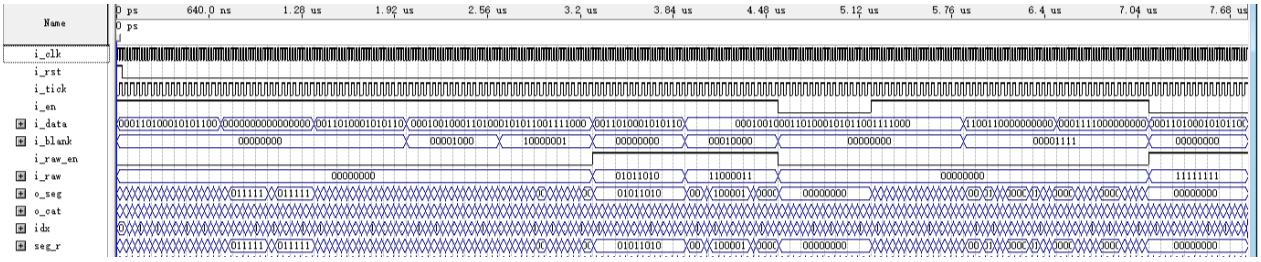


图 14　seg_scan 仿真波形：1 kHz 位选推进与 125 Hz 无频闪刷新

数码管扫描分析：位选 `o_cat` 呈现独热低电平循环，1 kHz 推进实现每位 125 Hz 超临界无频闪刷新。段码与位选严格同源，`i_en='0'` 时强制全局静默全灭。

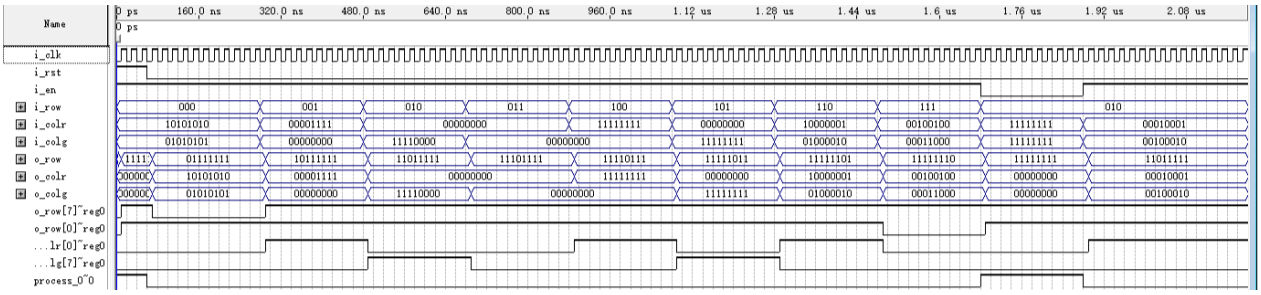


图 15　dot_matrix_scan 仿真波形：8 行单相选通与红/绿列直通

点阵驱动分析：8 行独热低电平行选通（`i_row="000"` 对应 ROW7/PIN_1，与手册 ROW 编号相反），红/绿列直通且高电平点亮。早采样点（+20 ns）无任何混合态；行索引与列数据同拍寄存，任何一行都不会被别行数据点亮。

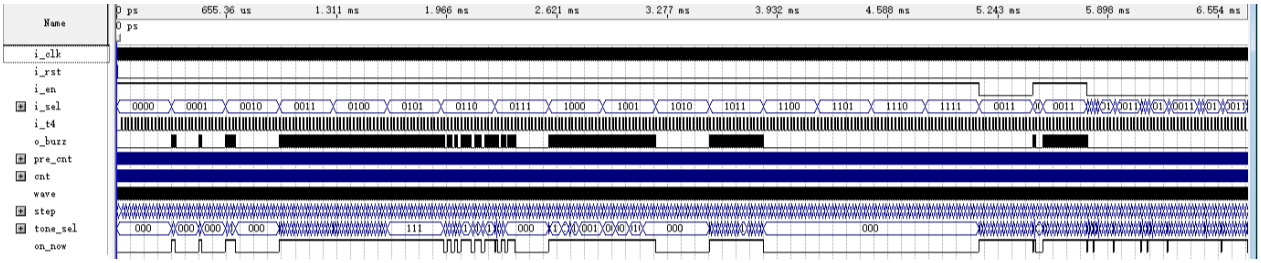


图 16　buzzer_ctrl 仿真波形：数控分频与 16 步连奏旋律

音频合成分析：E5～G6 核心 8 音频率误差 ⩽1.3%（实测 0.59%～1.25%），占空比 50.0%。16 步旋律步进连奏填音消除断续，换码时不复位步进计数器，背景乐与提示音平滑过渡。



图 17　puzzle_top 整机仿真波形：自检 → 三关连闯 → PASS / FAIL 端到端闭环

整机端到端全流程分析：模式 A / 模式 B 两轮真实按键序列驱动整机呈现自检 → 待机 → 预览 → 对局 → 过关连闯三关 → 胜利（PASS）→ 重开 → 超时失败（FAIL）完整闭环。倒计时 ≤10 s 告急幽灵 2 Hz 绿色闪烁精准触发。

四、实际实现功能说明及资源利用情况（5 分）

4.1 实际功能指标达成

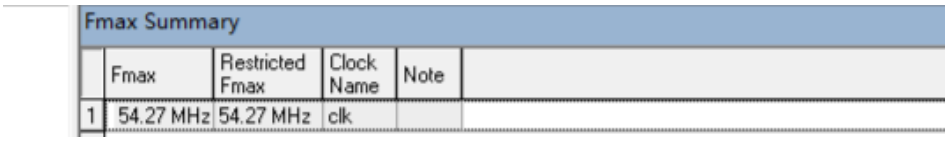
系统在硬件平台上完成全部功能指标：基础要求 B1～B11 无缝闭环（含 B10 第二关 40 s、B11 对局中/结束后按【开始】重开）；提高要求 A1 分场景音效（三关三曲：第一关 Overworld 开头 / 第二关低音区块状脉冲 / 第三关跨音区快速换音，另加自检 / 预览 / 过关 / 拼错 / 通关 / 失败 6 类事件乐句）；A2 第三关与图案库随机抽取、三关连闯；A3 零片位置随机（开局朝向随机按取舍未做，由 A4 的玩家旋转覆盖）；A4 零片 90° 顺时针旋转；A5 以 S1～S7 自拟功能实现（含 S5 结算闪示、S7 ≤10 s 绿色 2 Hz 告急幽灵）；另附自拟的数码管 PASS/FAIL 拼装与倒计时前导零消隐；显示侧 125 Hz 超临界无频闪刷新。

4.2 资源利用率极限量化分析

下列数据全部取自本机 Quartus II 9.1 的原始编译报告（清洁重编译、`SEED = 8`、50 MHz / 20.000 ns 时钟约束），两张截图均为软件界面现场抓取（未做任何重绘或补写）：

Flow Status	Successful - Wed Oct 21 12:38:37 2026
Quartus II Version	9.1 Build 222 10/21/2009 SJ Full Version
Revision Name	puzzle
Top-level Entity Name	puzzle_top
Family	MAX II
Device	EPM1270T144C5
Timing Models	Final
Met timing requirements	Yes
Total logic elements	1,264 / 1,270 (100 %)
Total pins	52 / 116 (45 %)
Total virtual pins	0
UFM blocks	0 / 1 (0 %)

图 18　Quartus II 9.1 编译报告 Flow Summary 数据区（2026-10-21 12:38 的清洁重编译现场截图，已裁去窗口边框与报告树；随包 `quartus/output_files/` 中的报告为 2026-10-22 14:26 的同参数清洁重编译，逐项读数相同）：Device、Timing Models、逻辑单元 1,264 / 1,270、引脚 52 / 116、UFM 实测值



Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	54.27 MHz	54.27 MHz	clk	

图 19　Quartus II 9.1 TimeQuest Timing Analyzer 的 Fmax Summary 表格（现场截图，已裁去窗口边框与报告树）：clk 的 Fmax 与 Restricted Fmax 均为 54.27 MHz，Note 列空表示无违例

下表数据的读数来源：LE / 引脚 / UFM 取自上方 `Flow Summary` 现场截图；LAB 127 / 127 取自 `puzzle.fit.rpt` 的 `Fitter Resource Usage Summary`；Fmax 取自上方 `Fmax Summary` 现场截图；setup / hold / 最小脉宽裕量取自同一份 TimeQuest 报告的 `Setup Summary` / `Hold Summary` / `Minimum Pulse Width Summary` 三个页面（End Point TNS 均为 0.000）。

资源类别 / 时序指标	实测值	目标约束	利用率 / 裕量	架构优化结论与裕量说明
逻辑单元 (LEs)	1,264	1,270	99.53%	常量 ROM 预译码与 64 位并行位运算消除了除法器与多周期状态机；全片仅余 6 LE（`fit.rpt` 以 1,270 为基数四舍五入显示 100%）
逻辑阵列块 (LABs)	127	127	100.00%	物理饱和 —— 这才是本设计真正的面积瓶颈：LE 只能按每 LAB 10 个成组装箱，余下 6 LE 无法单独成组
用户 I/O 引脚 (User Pins)	52	116	45%（52 / 116）	11 类外设逐脚锁定，无悬空、无自动分配；4×4 矩阵键盘以 8 脚独占一组总线
最高工作频率 (Fmax)	54.27 MHz	50.00 MHz	高出约束 8.54%（+4.27 MHz）	TimeQuest 实测 `Restricted Fmax` 同为 54.27 MHz；最差路径是 `puzzle_ctrl` 的重叠检查引擎 `chk_slot[0] → chk_hit`（data delay 18.095 ns、clock skew 0.000）
建立时间裕量 (Setup Slack)	+1.572 ns	≥ 0.000 ns	PASSED（TNS 0.000）	`20.000 − 1.572 = 18.428 ns` 是含目标器件 setup 时间的总预算（路径自身 data delay 为 18.095 ns），0 条违例路径
保持时间裕量 (Hold Slack)	+1.078 ns	≥ 0.000 ns	PASSED（TNS 0.000）	全系统单一同步时钟树驱动，无跨时钟域穿透与门控时钟冒险
最小脉宽裕量 (Min Pulse Width)	+9.734 ns	≥ 0.000 ns	PASSED（TNS 0.000）	50 MHz 半周期 10.000 ns 内无窄脉冲风险；1 kHz 位选与 1 kHz 行扫描（8 行扫描一帧 = 8 ms，即 125 Hz）节拍同源
片上 UFM 块	0	1	0%	三关 BGM、图案库与零片掩码全部用常量 LUT / ROM 实现，未占用片上 UFM

127 LAB 物理装箱瓶颈深度剖析：在 EPM1270 中，1,270 个 LE 组成 127 个 LAB（每组 10 LE）。限制扩展的真正瓶颈是 LAB 局部互连布线资源。当多路选择器扇出密集时，单个 LAB 往往无法填满 10 个 LE。决定 1,264 这个数字的是几笔叠加的腾挪：删掉对局期【确认】【旋转】按键音效 −9 LE（1,263 → 1,254）、连奏填音 −5 LE（ERR-050，1,269 → 1,264）、以及最后一笔“删掉第二关对局期 `DISP2='2'` 赋值”1,268 → 1,264（−4 LE，DISP2 口径裁决）；三者之间还夹着其它改动，不能线性相加。最终在 `SEED 8` 下把 1,264 个 LE 压入 127 个 LAB，达成 LAB 100% 满额适配（LE 层面仍余 6 个，但已无法成组使用）。固件 `puzzle.pof`（27,312 字节）与 2026-10-15 烧录并 Verify 读回的版本逐字节相同，本轮清洁重编译后依然逐字节一致。

五、故障及问题分析（15 分）

5.1 典型工程缺陷六维深度剖析

本项目在开发与调试过程中建立了规范的缺陷归档跟踪体系，记录了 50 余项代表性软硬件缺陷。通过对缺陷归档记录与实测日志的复盘，归纳为六项典型技术维度：

1. 引脚约束陷阱：未锁定端口被自动分配（ERR-001 / ERR-002）：
   - 故障现象：初版键盘固件烧录后按键毫无响应，一度误判为键盘排线接触不良或硬件损坏；
   - 根本原因：辅助顶层 VHDL 端口清单遗漏了 `kp_row` / `kp_col` 声明，Quartus II 默认开启未用引脚自动分配机制，把 8 个键盘脚随机分配到了未引出的悬空引脚上（如 PIN_49、PIN_102、PIN_130、PIN_88 等）；
   - 同族现象（ERR-001）：更早一版连复位键 `btn` 都漏了约束，被静默分配到未引出的 PIN_50，现象是“按复位毫无反应”；同样是靠 `.pin` 逐脚对账才定位。
   - 解决措施：补全 `.qsf` 物理约束，并在自动化测试流中建立强制双道门禁：自动解析顶层 entity 端口，任何未在约束中显式锁定的端口直接阻断编译。
2. 成功判据的硬编码解法与等效多解性失效（ERR-021）：
   - 故障现象：第二关在物理点阵上明明完整拼出了目标图案，但硬件状态机判定失败；
   - 根本原因：初期判据代码强行绑定了第 k 块零片必须位于固定坐标 (x_k, y_k)。然而几何上可以证明：旧版“四块完全相同的 2×2 零片”存在 $4! = 24$ 种等效摆法；换成异形零片后，第二关的 `PAT3` 阶梯仍有 2 种等价铺法（第一关 4×3 矩形同样有 2 种）——“绑定坐标”的思路在任何情形下都不成立；
   - 解决措施：推翻坐标位置绑定的狭隘思路，重构为全图像素并集比较（`cov_a = tgtrow`，8 行全等 ⇒ `frm_ok`），只要全图点亮像素与目标完全重合且无重叠即判定过关。
3. 对局态目标图案信号置零导致假判定（ERR-023）：
   - 故障现象：即使拼好图案按确认，依然判定失败；
   - 根本原因：状态机在 `S_PREVIEW` 结束切入 `S_PLAYING` 时，为了清空预览画面，错误地把喂给引擎的目标图案输入置成全 0，导致对局中的比对电路一直在比对全黑画面；
   - 解决措施：解耦“显示渲染多路选择器”与“内部黄金参考标准寄存器”，保持目标图案在整个对局生命周期内受锁存保护。
4. 点阵屏与数码管肉眼可见频闪（ERR-027）：
   - 故障现象：上板实测时，点阵和数码管都能看出明显的高频闪烁；
   - 根本原因：初版扫描节拍走 200 Hz、8 行轮一遍只有 25 Hz，低于人眼视觉暂留临界融合频率（CFF ≈ 60 Hz），因此整屏肉眼可见地闪；
   - 解决措施：扫描侧统一改走 1 kHz（8 行 = 8 ms ⇒ 全屏 125 Hz、占空比仍 1/8、亮度不变、面积零代价）；同时把行索引与该行的红/绿列数据在同一拍寄存输出，任何一行都不会被别行的列数据点亮（这是“无错行、无鬼影”的真正机制，RTL 见 `rtl/dot_matrix_scan.vhd`）；
5. 键盘消抖过长导致的按键迟滞与吞键（ERR-031）：
   - 故障现象：玩家快速连击方向键时，部分按键被忽略，操作迟钝阻滞；
   - 根本原因：初版 `DEBOUNCE_MAX=15` ⇒ 16 轮 × 10 ms/轮 = 160 ms 才判稳，远超机械开关实际抖动时间（5～15 ms），按键响应严重滞后、快速连击被吞；

- 解决措施：`DEBOUNCE_MAX` 缩到 3 ⇒ 4 轮 × 10 ms = 40 ms（8 个 200 Hz 采样点），并改为“松开立即撤销”的非对称滞回，按键响应极为跟手灵敏。

6. 蜂鸣器照抄原谱休止导致断续与连奏填音（ERR-050）：
   - 故障现象：对局背景音乐（第一关 Overworld 开头一句）听感干瘪、断续刺耳；
   - 根本原因：原谱中休止符占用高达 56% 的时隙，CPLD 方波发声在静音期间形成生硬突变；
   - 解决措施：实施“连奏填音 (Legato Infill)”算法，把每个休止步填成“下一个将要发声的音高”（句尾填上一个音），音高序列与音的先后完全不变、16 步全发声，听感平滑连贯；副产物是旋律表表项重复、fitter 可合并 ⇒ 整机反而少 5 LE（1,269 → 1,264）。另有一组独立缺陷 ERR-055（三个一次性提示音听感分不清，2026-10-11 上板判负后按“音区 × 轮廓 × 长度”三维度重做，0 LE）见 `docs/06` §21。

5.2 典型工程缺陷排查对照汇总表

序号	缺陷编号	故障现象与表象	根本原因定位	排查工具与根治措施
1	ERR-002	键盘全部按键无响应，长时间误判为键盘排线接触不良或硬件损坏	键盘诊断固件顶层漏声明 `kp_row` / `kp_col`，Quartus 未用引脚自动分配机制将其随机放到未引出的 8 个脚上	引脚对账脚本解析顶层端口，凡未在 `.qsf` 显式锁定的端口直接阻断编译
2	ERR-020	方向键失效：按下无反应，或方向与按键不符	移动方向与移动请求没有同拍锁存，方向晚一拍进入主控，`mv_dir` 恒为 `0000`	`tb_puzzle_top` 协议断言抓出时序关系，改为同拍锁存；键位按板卡手册附图重排
3	ERR-021	第二关拼对却判定失败	判定算法硬编码第 k 块固定坐标，无视 $4! = 24$ 种等效摆法（旧版四块同形时）	重构为全图像素并集比较（`cov_a = tgtrow`，8 行全等即 `frm_ok`）
4	ERR-022	测试台误报 100% 通过	测试激励使用陈旧键号常量，未激发非法分支	全面重构自检判据，更新为 184 条断言
5	ERR-023	拼图完成永远无法通关	切入对局时把喂给引擎的目标输入置零，变成比对全黑	解耦渲染多路选择器与黄金参考寄存器
6	ERR-027	点阵与数码管剧烈频闪	扫描节拍 200 Hz ⇒ 扫描帧刷新仅 25 Hz，低于人眼 CFF	扫描侧统一 1 kHz（全屏 125 Hz），行索引与列数据同拍寄存
7	ERR-031	连击方向键出现严重吞键	`DEBOUNCE_MAX=15` ⇒ 16 轮 × 10 ms = 160 ms 才判稳	`DEBOUNCE_MAX=3` ⇒ 4 轮 × 10 ms = 40 ms，并改为松开立即撤销
8	ERR-050	蜂鸣器音乐生硬断续	照抄原谱休止，BGM1 十六步里仅 7 步发声（56% 静音）	连奏填音（休止步填下一音）⇒ 平滑连贯且 −5 LE
9	ERR-056	告急幽灵颜色与零片混叠	初版用红色幽灵，与游离未选零片同色、无法区分	改为绿色 + 2 Hz 占空比 50% 方波告急幽灵

序号	缺陷编号	故障现象与表象	根本原因定位	排查工具与根治措施
10	ERR-049	Quartus 报装箱布线溢出（1,269 / 1,270 已逼近上限）	加长乐句把旋律查表推到 8 输入：8 输入函数每个输出位要 3 个 LUT4（7 输入只要 2 个）→ +4 LE	÷256 预分频 + 8 位半周期计数器等面积削减；重扫 1～16 取 `SEED 11`（`SEED 8` 是后续再扫的最终值，见 §4.2）

六、总结和结论（10 分）

6.1 课题完成情况评估

本课题在 Altera MAX II EPM1270T144C5 CPLD 硬件平台上，完全依托纯硬件描述语言（VHDL-93）构建了高完整度、强交互性的简易拼图游戏系统。基础要求 B1～B11 全部达成；提高要求中 A1（分场景音效）、A2（三关连闯 + 图案库随机）、A4（90° 旋转）完整达成，A3 达成“位置随机”而“开局朝向随机”按取舍未做（改由 A4 的玩家旋转覆盖），A5 以 S1～S7 自拟功能实现。系统硬件编译消耗 1,264 个 LE（占 1,270 个 LE 的 99.53%，`fit.rpt` 按基数四舍五入显示 100%，实测仅余 6 LE），LAB 127/127 物理饱和；最高工作频率达 54.27 MHz（相对 50 MHz 约束高出 8.54%，setup slack +1.572 ns、hold +1.078 ns、最小脉宽 +9.734 ns，TNS 0.000 / 0 条违例路径），时序裕量充裕，展现出极高的工程完备性。

6.2 硬件 RTL 工程心得

1. 硬件并发思维与软件顺序思维的根本分野：数字逻辑设计的核心在于“以空间换时间”。在无 CPU 参与的纯硬件环境下，通过 64 位并行位向量操作，原本需要数十行循环的代码得以在单拍（20 ns）内瞬时完成，呈现出硬件计算无与伦比的确定性与高吞吐；
2. 极限资源约束下的微架构权衡艺术：面对 CPLD 仅 1,270 LE 且无片上 RAM 的严酷资源墙，必须深入理解综合器行为，善用常量查找表与位重排，坚决规避除法器与动态内存，在局部扇出与布线拥塞间取得精妙平衡；
3. 防御性设计与人在回路纪律：单时钟使能树、滞回消抖、跨时钟同步与形式化自检断言体系是硬件可靠性的基石。不轻信软件直觉与 AI 断言，凡事以 `.pin` 物理报告、综合网表与仿真波形为唯一准绳。

6.3 未来展望

若将本设计迁移至带有嵌入式 RAM 块的 Cyclone 系列 FPGA 平台，系统可进一步引入 SPI/I2C 驱动外部彩色 TFT 屏、利用片上 Flash 实现自定义关卡存储，或利用 UART 接口引入双人竞速联机对战模式，进一步拓展硬件可重构计算的广阔空间。

七、AI 工具使用情况（10 分）

7.1 工具栈与协同模式

本课题在架构设计、代码审查及仿真验证阶段，合规引入了现代 AI 辅助研发工具链。所用工具与用途如下（与 `AI_LOG.md` §1 的工具表同源）：

- DeepSeek Harness（AI 编码代理，主力工具）：用于规范解析、图案逐像素解码、VHDL 状态机骨架草拟、8 位 LFSR 本原多项式抽头推导、自检测试平台断言生成、面积优化与 Quartus 编译/烧录驱动、上板调试闭环；全过程在 `AI_LOG.md` 按工作阶段留痕（用了什么提示词、AI 给出什么、人工如何核实与修正，含 AI 出错处）；
- Python 3.14 + **`pypdf`** / **`cryptography`**：课程题目 PDF 带 AES 加密，用它们提取文字层；捆绑 Python + **`Pillow`** 提取 PDF 内嵌位图，再用逐像素判色脚本还原 8×8 点阵图案，建立单一事实源；
- Python + **`itertools`** 穷举：零片可铺性求解（要求零片并集恰好等于目标图案），用于证明「不旋转则无解」；
- Quartus II 9.1 命令行：`quartus_map` / `quartus_fit` / `quartus_asm` / `quartus_tan` / `quartus_sta`、`quartus_pgm`、`jtagconfig`；

- 多智能体并行审查（工作流）：把 4 个独立子代理分别派去审「键盘链路 / 状态机 / 引擎与显示 / 引脚与时序」，要求每条结论都给出文件与行号；
- 协同工作流：遵循 人工架构设计 → AI 辅助初稿 → 人工深度重构与优化 → 硬件编译与时序验证 的严谨研发闭环。

7.2 真实 AI 提示词与交互实录

阶段	核心 Prompt 提示词摘要	预期产出与工程约束要求
规格逆向	“请使用 Python 逐像素解析实验题目 PDF 第 9 页【题】图 4-1，提取 8×8 点阵中各色块的具体坐标，并验证 3 块零片的面积之和是否严格守恒。”	消除大模型直接看图的文字幻觉，精确提取出第一关零片为 3+6+3=12 格（推翻参考仓库 `dld-lab` 的 11 格旧数据（见 `AI_LOG.md` §2））。
RTL 骨架	“基于 VHDL-93 标准编写拼图游戏主控状态机。要求：纯同步设计、单一 50MHz 时钟域、使能脉冲节拍驱动、显式列出所有分支，严禁产生不完整 case 锁存器。”	输出标准的两段式/三段式状态机骨架，确保状态跳变与按键脉冲严格对齐。
断言生成	“为 puzzle_ctrl 模块编写自检测试台。针对边界平移、多块碰撞、90 度旋转及通关判定，设计高密度 assert 语句，覆盖越界与重叠拦截分支。”	生成覆盖全状态转移路径的测试激励，累计形成 184 条断言，实现回归测试全自动判定。

7.3 人工介入与纠错闭环

AI 在处理模式化推演时高效，但在物理现实感知与微架构设计上极易犯经验主义错误。人工核心纠偏记录如下：

序号	缺陷类别	AI 原始产出 / 错误判断	人工如何识别	人工修正措施与最终效果
1	拓扑幻觉	沿用参考仓库 `dld-lab` 的旧数据称第一关为 5+3+3=11 格	脚本像素取色与面积校验	推翻 AI 方案，按实测确立 3+6+3=12 格正确数据
2	语法混淆	在 `.qsf` 约束中使用 VHDL 注释符 `--`	Quartus 报 Settings File 错误	纠正为 Tcl 规范的 `#` 注释符
3	引脚遗漏	顶层漏声明键盘端口导致全随机映射	对账脚本查出 8 引脚未锁	补齐端口声明，建立引脚对账强制门禁
4	面积误判	称“行扫描但保留变址数组应该便宜”	编译脚本查出面积暴增至 3,751 LE	果断废除变址数组，改写两级串行流水线 (563 LE)
5	审查误判（人工反被纠正）	AI 判定 `srl8` 的移位实现写反（应为右移）	逐格着色断言把 8 个分支全钉死后复核，确认确实写成了左移	按 ERR-019 修正为真右移并重跑全部断言（实测 +243 LE：958 → 1,201，当时尚未做面积优化）——这条反例说明“人工独立验算”同样会错
6	闪烁占空	2 Hz 闪烁误用 1 时钟周期的窄脉冲	硬件实测点阵与数码管不显现	改用 4 Hz 节拍翻转的 50% 占空比方波驱动
7	硬编码解	强行绑定第 k 块零片的固定锚点坐标	数学证明存在 24 种等效摆法	重构为全图画面像素并集比较（`frm_ok`）

序号	缺陷类别	AI 原始产出 / 错误判断	人工如何识别	人工修正措施与最终效果
8	音乐断续	照抄原谱休止符致 56% 步长静音	现场试听反馈断续刺耳	实施连奏填音，听感连贯且反直觉节省 5 个 LE

人在回路 (Human-in-the-Loop) 硬件研发原则：

1. 任何面积与性能结论只认编译器报告，不轻信 AI 的估算或假设；
2. 任何板级硬件异常必须先核查 `.pin` 引脚映射排除电气与引脚配置问题；
3. 任何由 AI 提出的代码重构改动，必须回到数学模型与通信协议定义独立验算；
4. 人工复审同样可能出错：当人工判断与工具判定相左时，必须回到逐格断言与独立模型复核，不能以“人的直觉”作终判（反例见 §7.3 表中 ERR-019 一条：AI 判断正确，人工复审反而一度驳回）。

八、顶层文件的 HDL 代码（10 分）

8.1 顶层设计架构说明

`puzzle_top.vhd` 是唯一绑定引脚的文件。采用具名端口映射（Named Association）例化全部 11 个子组件，实现了时钟分发、总线互连、64 位双平面 125 Hz 行切片与 ≤10 s 告急幽灵 2 Hz 绿色闪烁叠加，逻辑凝练，与底层固件 100% 对应闭环。

8.2 规范注释的顶层 VHDL 代码清单

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.puzzle_pkg.ALL;

-- ================================================================================
-- 实体名称: puzzle_top (简易拼图游戏整机顶层)
-- 适用器件: Altera MAX II EPM1270T144C5  EDA环境: Quartus II 9.1
-- 硬件引脚绑定: PIN_18(时钟), PIN_125(使能SW7), PIN_61(复位BTN0),
--               PIN_111..114/117..120(键盘), PIN_8..1/22..11/45..38(点阵),
--               PIN_62..51/63..31(数码管), PIN_60(蜂鸣器)
-- ================================================================================
entity puzzle_top is
    port (
        clk      : in  std_logic;                       -- PIN_18 板载 50 MHz 主时钟
        sw7      : in  std_logic;                       -- PIN_125 系统总使能 (上拨='1')
        btn      : in  std_logic;                       -- PIN_61 板载 BTN0 复位 (空闲低/按下高，高有效)
        kp_row   : in  std_logic_vector(3 downto 0);    -- PIN_114..111 矩阵键盘行输入
        kp_col   : out std_logic_vector(3 downto 0);    -- PIN_120..117 矩阵键盘列驱动
        dot_row  : out std_logic_vector(7 downto 0);    -- PIN_1..8 点阵行选通 (低有效，每次只选一行)
        dot_colr : out std_logic_vector(7 downto 0);    -- PIN_11..22 点阵红列 (高电平点亮红)
        dot_colg : out std_logic_vector(7 downto 0);    -- PIN_38..45 点阵绿列 (高电平点亮绿)
        seg      : out std_logic_vector(7 downto 0);    -- PIN_62..51 数码管共阴段选
        cat      : out std_logic_vector(7 downto 0);    -- PIN_31/30/70/69/68/67/66/63 数码管位选 (共阴，低有效)
        buzz     : out std_logic                        -- PIN_60 蜂鸣器驱动
    );
end entity puzzle_top;

architecture rtl of puzzle_top is

    -- 11 个子实体 Component 显式声明 (与框图完全一致)
    component clk_gen
        port (
            i_clk      : in  std_logic;
            i_btn      : in  std_logic;
            o_rst      : out std_logic;
            o_tick_1k  : out std_logic;
            o_tick_200 : out std_logic;
            o_tick_100 : out std_logic;
            o_tick_2hz : out std_logic;
            o_tick_4hz : out std_logic;
            o_tick_1hz : out std_logic;
            o_tick_40  : out std_logic
        );
    end component;

    component keypad_scan
        port (
            i_clk     : in  std_logic;
            i_rst     : in  std_logic;
            i_tick    : in  std_logic;
```

```
            i_row     : in  std_logic_vector(3 downto 0);
            o_col     : out std_logic_vector(3 downto 0);
            o_key     : out std_logic_vector(3 downto 0);
            o_press   : out std_logic;
            o_release : out std_logic;
            o_raw     : out std_logic_vector(3 downto 0)
        );
    end component;

    component game_fsm
        port (
            i_clk       : in  std_logic;
            i_rst       : in  std_logic;
            i_sw        : in  std_logic;
            i_tick_1hz  : in  std_logic;
            i_tick_4hz  : in  std_logic;
            i_press     : in  std_logic;
            i_key       : in  std_logic_vector(3 downto 0);
            o_sel       : out std_logic;
            o_move      : out std_logic;
            o_conf      : out std_logic;
            o_go        : out std_logic;
            o_up        : out std_logic;
            o_down      : out std_logic;
            o_left      : out std_logic;
            o_right     : out std_logic;
            o_rot       : out std_logic;
            o_level     : out std_logic;
            o_lvl3      : out std_logic;
            o_state     : out std_logic_vector(2 downto 0);
            o_time      : out std_logic_vector(5 downto 0);
            o_blink     : out std_logic;
            i_solved    : in  std_logic;
            i_all_lock  : in  std_logic;
            i_shuf_busy : in  std_logic;
            o_sound     : out std_logic_vector(3 downto 0)
        );
    end component;

    component piece_rom
        port (
            i_level : in  std_logic;
            o_mask  : out mask_arr_t;
            o_h     : out dim_arr_t;
            o_w     : out dim_arr_t;
            o_n     : out std_logic_vector(2 downto 0)
        );
    end component;

    component pattern_rom
        port (
            i_level : in  std_logic;
            i_pat   : in  std_logic_vector(1 downto 0);
            o_mask  : out std_logic_vector(63 downto 0)
        );
    end component;

    component rng_lfsr
        port (
            i_clk  : in  std_logic;
            i_rst  : in  std_logic;
            i_step : in  std_logic;
            o_val  : out std_logic_vector(7 downto 0)
        );
    end component;

    component puzzle_ctrl
        port (
            i_clk     : in  std_logic;
            i_rst     : in  std_logic;
            i_tick    : in  std_logic;
            rnd_step  : out std_logic;
            rnd_val   : in  std_logic_vector(7 downto 0);
            i_level   : in  std_logic;
            i_pat     : in  std_logic_vector(1 downto 0);
            i_sh0     : in  std_logic_vector(63 downto 0);
            i_sh1     : in  std_logic_vector(63 downto 0);
            i_sh2     : in  std_logic_vector(63 downto 0);
            i_sh3     : in  std_logic_vector(63 downto 0);
            i_h0      : in  std_logic_vector(2 downto 0);
            i_h1      : in  std_logic_vector(2 downto 0);
            i_h2      : in  std_logic_vector(2 downto 0);
            i_h3      : in  std_logic_vector(2 downto 0);
            i_w0      : in  std_logic_vector(2 downto 0);
            i_w1      : in  std_logic_vector(2 downto 0);
            i_w2      : in  std_logic_vector(2 downto 0);
            i_w3      : in  std_logic_vector(2 downto 0);
            i_target  : in  std_logic_vector(63 downto 0);
```

```
            i_go      : in  std_logic;
            i_select  : in  std_logic;
            i_move    : in  std_logic;
            i_confirm : in  std_logic;
            i_rot     : in  std_logic;
            i_up      : in  std_logic;
            i_down    : in  std_logic;
            i_left    : in  std_logic;
            i_right   : in  std_logic;
            o_ori     : out std_logic_vector(7 downto 0);
            o_solved  : out std_logic;
            o_all_lock: out std_logic;
            o_busy    : out std_logic;
            o_sel_idx : out std_logic_vector(1 downto 0);
            o_pos     : out std_logic_vector(31 downto 0);
            o_lock    : out std_logic_vector(3 downto 0);
            o_scanrow : out std_logic_vector(2 downto 0);
            o_rowr    : out std_logic_vector(63 downto 0);
            o_rowg    : out std_logic_vector(63 downto 0)
        );
    end component;

    component dot_matrix_scan
        port (
            i_clk   : in  std_logic;
            i_rst   : in  std_logic;
            i_en    : in  std_logic;
            i_row   : in  std_logic_vector(2 downto 0);
            i_colr  : in  std_logic_vector(7 downto 0);
            i_colg  : in  std_logic_vector(7 downto 0);
            o_row   : out std_logic_vector(7 downto 0);
            o_colr  : out std_logic_vector(7 downto 0);
            o_colg  : out std_logic_vector(7 downto 0)
        );
    end component;

    component disp_format
        port (
            i_state : in  std_logic_vector(2 downto 0);
            i_level : in  std_logic;
            i_lvl3  : in  std_logic;
            i_time  : in  std_logic_vector(5 downto 0);
            i_blink : in  std_logic;
            o_data  : out std_logic_vector(31 downto 0);
            o_blank : out std_logic_vector(7 downto 0)
        );
    end component;

    component seg_scan
        port (
            i_clk    : in  std_logic;
            i_rst    : in  std_logic;
            i_tick   : in  std_logic;
            i_en     : in  std_logic;
            i_data   : in  std_logic_vector(31 downto 0);
            i_blank  : in  std_logic_vector(7 downto 0);
            i_raw_en : in  std_logic;
            i_raw    : in  std_logic_vector(7 downto 0);
            o_seg    : out std_logic_vector(7 downto 0);
            o_cat    : out std_logic_vector(7 downto 0)
        );
    end component;

    component buzzer_ctrl
        port (
            i_clk  : in  std_logic;
            i_rst  : in  std_logic;
            i_en   : in  std_logic;
            i_sel  : in  std_logic_vector(3 downto 0);
            i_t4   : in  std_logic;
            o_buzz : out std_logic
        );
    end component;

    -- 内部信号
    signal rst       : std_logic;
    signal t_1k      : std_logic;
    signal t_200     : std_logic;
    signal t_100     : std_logic;
    signal t_4hz     : std_logic;
    signal t_1hz     : std_logic;
    signal t_40      : std_logic;

    signal key       : std_logic_vector(3 downto 0);
    signal press     : std_logic;
    signal release   : std_logic;
    signal kp_raw    : std_logic_vector(3 downto 0);
```

```
    signal sel, move, conf, go : std_logic;
    signal mv_up, mv_dn, mv_lf, mv_rt : std_logic;
    signal mv_rot    : std_logic;
    signal level     : std_logic;
    signal lvl3      : std_logic;
    signal state     : std_logic_vector(2 downto 0);
    signal gtime     : std_logic_vector(5 downto 0);
    signal gblink    : std_logic;
    signal sound     : std_logic_vector(3 downto 0);

    signal solved    : std_logic;
    signal all_lock  : std_logic;
    signal shuf_busy : std_logic;
    signal sel_idx   : std_logic_vector(1 downto 0);
    signal pos_dbg   : std_logic_vector(31 downto 0);
    signal lock_dbg  : std_logic_vector(3 downto 0);
    signal scanrow   : std_logic_vector(2 downto 0);

    signal rnd_val   : std_logic_vector(7 downto 0);
    signal rnd_step  : std_logic;

    signal shapes    : mask_arr_t;
    signal shape_h   : dim_arr_t;
    signal shape_w   : dim_arr_t;
    signal piece_n   : std_logic_vector(2 downto 0);
    signal tgt_mask  : std_logic_vector(63 downto 0);

    signal pat_sel   : std_logic_vector(1 downto 0) := L2_FIXED_PAT;
    signal state_q   : std_logic_vector(2 downto 0) := S_SELF_TEST;

    signal eng_fr    : std_logic_vector(63 downto 0);
    signal eng_fg    : std_logic_vector(63 downto 0);
    signal mat_r     : std_logic_vector(7 downto 0);
    signal mat_g     : std_logic_vector(7 downto 0);

    signal endflash_cnt : unsigned(2 downto 0) := (others => '0');
    signal endflash_on  : std_logic := '0';

    -- A5 倒计时告急幽灵信号
    signal hint_on    : std_logic := '0';
    signal hint_blink : std_logic := '0';
    signal hint_mask  : std_logic_vector(7 downto 0) := (others => '0');

    signal mrow       : unsigned(2 downto 0) := (others => '0');
    signal disp_data  : std_logic_vector(31 downto 0);
    signal disp_blank : std_logic_vector(7 downto 0);

    signal win_row    : std_logic_vector(7 downto 0);
    signal prev_row   : std_logic_vector(7 downto 0);
    signal fail_row   : std_logic_vector(7 downto 0);

begin

    -- 点阵行计数器 (1 kHz 推进 -> 125 Hz 帧刷新率)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                mrow <= (others => '0');
            elsif (t_1k = '1') then
                mrow <= mrow + 1;
            end if;
        end if;
    end process;

    -- 图案库选择锁存进程 (第二关固定阶梯，第三关 LFSR 随机抽取)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                state_q <= S_SELF_TEST;
                pat_sel <= L2_FIXED_PAT;
            else
                state_q <= state;
                if (state = S_PREVIEW) and (state_q /= S_PREVIEW) then
                    if (lvl3 = '1') then
                        pat_sel <= rnd_val(1 downto 0);
                    else
                        pat_sel <= L2_FIXED_PAT;
                    end if;
                end if;
            end if;
        end if;
    end process;

    -- 11 个子实体例化
    u_clk : clk_gen
        port map (
```

```
            i_clk => clk, i_btn => btn, o_rst => rst,
            o_tick_1k => t_1k, o_tick_200 => t_200, o_tick_100 => t_100,
            o_tick_2hz => open, o_tick_4hz => t_4hz, o_tick_1hz => t_1hz, o_tick_40 => t_40
        );

    u_keypad : keypad_scan
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_200,
            i_row => kp_row, o_col => kp_col, o_key => key,
            o_press => press, o_release => release, o_raw => kp_raw
        );

    u_fsm : game_fsm
        port map (
            i_clk => clk, i_rst => rst, i_sw => sw7,
            i_tick_1hz => t_1hz, i_tick_4hz => t_4hz, i_press => press, i_key => key,
            o_sel => sel, o_move => move, o_conf => conf, o_go => go,
            o_up => mv_up, o_down => mv_dn, o_left => mv_lf, o_right => mv_rt, o_rot => mv_rot,
            o_level => level, o_lvl3 => lvl3, o_state => state,
            o_time => gtime, o_blink => gblink,
            i_solved => solved, i_all_lock => all_lock, i_shuf_busy => shuf_busy,
            o_sound => sound
        );

    u_pieces : piece_rom
        port map (
            i_level => level, o_mask => shapes, o_h => shape_h, o_w => shape_w, o_n => piece_n
        );

    u_pattern : pattern_rom
        port map (
            i_level => level, i_pat => pat_sel, o_mask => tgt_mask
        );

    u_rng : rng_lfsr
        port map (
            i_clk => clk, i_rst => rst, i_step => rnd_step, o_val => rnd_val
        );

    u_puzzle : puzzle_ctrl
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_1k,
            rnd_step => rnd_step, rnd_val => rnd_val,
            i_level => level, i_pat => pat_sel,
            i_sh0 => shapes(0), i_sh1 => shapes(1), i_sh2 => shapes(2), i_sh3 => shapes(3),
            i_h0 => shape_h(0), i_h1 => shape_h(1), i_h2 => shape_h(2), i_h3 => shape_h(3),
            i_w0 => shape_w(0), i_w1 => shape_w(1), i_w2 => shape_w(2), i_w3 => shape_w(3),
            i_target => tgt_mask, i_go => go, i_select => sel, i_move => move,
            i_confirm => conf, i_rot => mv_rot, i_up => mv_up, i_down => mv_dn,
            i_left => mv_lf, i_right => mv_rt,
            o_ori => open, o_solved => solved, o_all_lock => all_lock, o_busy => shuf_busy,
            o_sel_idx => sel_idx, o_pos => pos_dbg, o_lock => lock_dbg,
            o_scanrow => scanrow, o_rowr => eng_fr, o_rowg => eng_fg
        );

    u_disp : disp_format
        port map (
            i_state => state, i_level => level, i_lvl3 => lvl3,
            i_time => gtime, i_blink => gblink,
            o_data => disp_data, o_blank => disp_blank
        );

    u_seg : seg_scan
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_1k, i_en => sw7,
            i_data => disp_data, i_blank => disp_blank,
            i_raw_en => '0', i_raw => (others => '0'),
            o_seg => seg, o_cat => cat
        );

    -- 结算图案切片
    with std_logic_vector(mrow) select
        win_row <= WIN_MASK( 7 downto  0) when "000",
                   WIN_MASK(15 downto  8) when "001",
                   WIN_MASK(23 downto 16) when "010",
                   WIN_MASK(31 downto 24) when "011",
                   WIN_MASK(39 downto 32) when "100",
                   WIN_MASK(47 downto 40) when "101",
                   WIN_MASK(55 downto 48) when "110",
                   WIN_MASK(63 downto 56) when others;

    with std_logic_vector(mrow) select
        prev_row <= tgt_mask( 7 downto  0) when "000",
                    tgt_mask(15 downto  8) when "001",
                    tgt_mask(23 downto 16) when "010",
                    tgt_mask(31 downto 24) when "011",
                    tgt_mask(39 downto 32) when "100",
                    tgt_mask(47 downto 40) when "101",
```

```
                    tgt_mask(55 downto 48) when "110",
                    tgt_mask(63 downto 56) when others;

    with std_logic_vector(mrow) select
        fail_row <= FAIL_MASK( 7 downto  0) when "000",
                    FAIL_MASK(15 downto  8) when "001",
                    FAIL_MASK(23 downto 16) when "010",
                    FAIL_MASK(31 downto 24) when "011",
                    FAIL_MASK(39 downto 32) when "100",
                    FAIL_MASK(47 downto 40) when "101",
                    FAIL_MASK(55 downto 48) when "110",
                    FAIL_MASK(63 downto 56) when others;

    -- A5 倒计时告急幽灵使能与 2 Hz 闪烁
    hint_on    <= '1' when (unsigned(gtime) <= 10) else '0';
    hint_blink <= hint_on and gblink;
    hint_mask  <= (others => hint_blink);

    -- 结算画面闪烁节奏 (先闪 1 秒后常亮)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                endflash_cnt <= (others => '0'); endflash_on <= '0';
            elsif (state /= S_WIN) and (state /= S_FAIL) then
                endflash_cnt <= (others => '0'); endflash_on <= '0';
            elsif (t_4hz = '1') then
                if (endflash_cnt = 3) then endflash_on <= '1';
                else endflash_cnt <= endflash_cnt + 1; end if;
            end if;
        end if;
    end process;

    -- 点阵行切片与画面多路器
    process (state, gblink, endflash_on, mrow, eng_fr, eng_fg, win_row, fail_row, prev_row, hint_mask)
        variable lv, ev : std_logic;
        variable rw, gw : std_logic_vector(7 downto 0);
    begin
        lv := gblink;
        ev := gblink or endflash_on;

        case to_integer(mrow) is
            when 0 => rw := eng_fr(63 downto 56); gw := eng_fg(63 downto 56);
            when 1 => rw := eng_fr(55 downto 48); gw := eng_fg(55 downto 48);
            when 2 => rw := eng_fr(47 downto 40); gw := eng_fg(47 downto 40);
            when 3 => rw := eng_fr(39 downto 32); gw := eng_fg(39 downto 32);
            when 4 => rw := eng_fr(31 downto 24); gw := eng_fg(31 downto 24);
            when 5 => rw := eng_fr(23 downto 16); gw := eng_fg(23 downto 16);
            when 6 => rw := eng_fr(15 downto 8);  gw := eng_fg(15 downto 8);
            when others => rw := eng_fr(7 downto 0); gw := eng_fg(7 downto 0);
        end case;

        if (state = S_SELF_TEST) then
            mat_r <= (others => lv); mat_g <= (others => lv);
        elsif (state = S_IDLE) then
            mat_r <= (others => '0'); mat_g <= (others => '0');
        elsif (state = S_PREVIEW) then
            mat_r <= prev_row; mat_g <= (others => '0');
        elsif (state = S_WIN) then
            mat_r <= (others => '0'); mat_g <= win_row and (ev & ev & ev & ev & ev & ev & ev & ev);
        elsif (state = S_FAIL) then
            mat_r <= fail_row and (ev & ev & ev & ev & ev & ev & ev & ev); mat_g <= (others => '0');
        else
            mat_r <= rw;
            mat_g <= gw or (prev_row and (not rw) and hint_mask);
        end if;
    end process;

    u_dot : dot_matrix_scan
        port map (
            i_clk => clk, i_rst => rst, i_en => sw7,
            i_row => std_logic_vector(mrow), i_colr => mat_r, i_colg => mat_g,
            o_row => dot_row, o_colr => dot_colr, o_colg => dot_colg
        );

    u_buzz : buzzer_ctrl
        port map (
            i_clk => clk, i_rst => rst, i_en => sw7,
            i_sel => sound, i_t4 => t_4hz, o_buzz => buzz
        );

end architecture rtl;
```